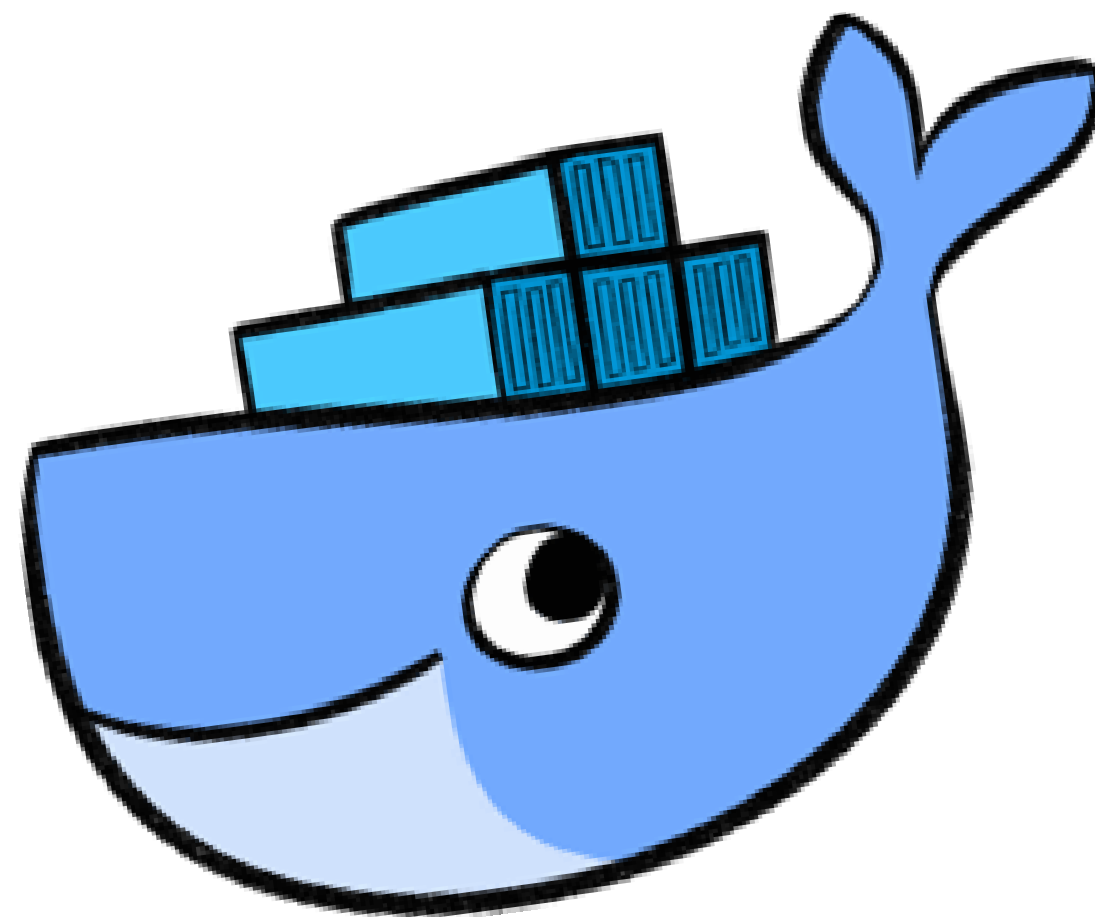




Minicurso de Docker

Aula 1 - Conceitos Fundamentais



Empacota a aplicação com tudo que ela precisa para ser executada (como dependências e configurações) em um contêiner isolado, garantindo que ela **funcione de forma idêntica em qualquer ambiente**.

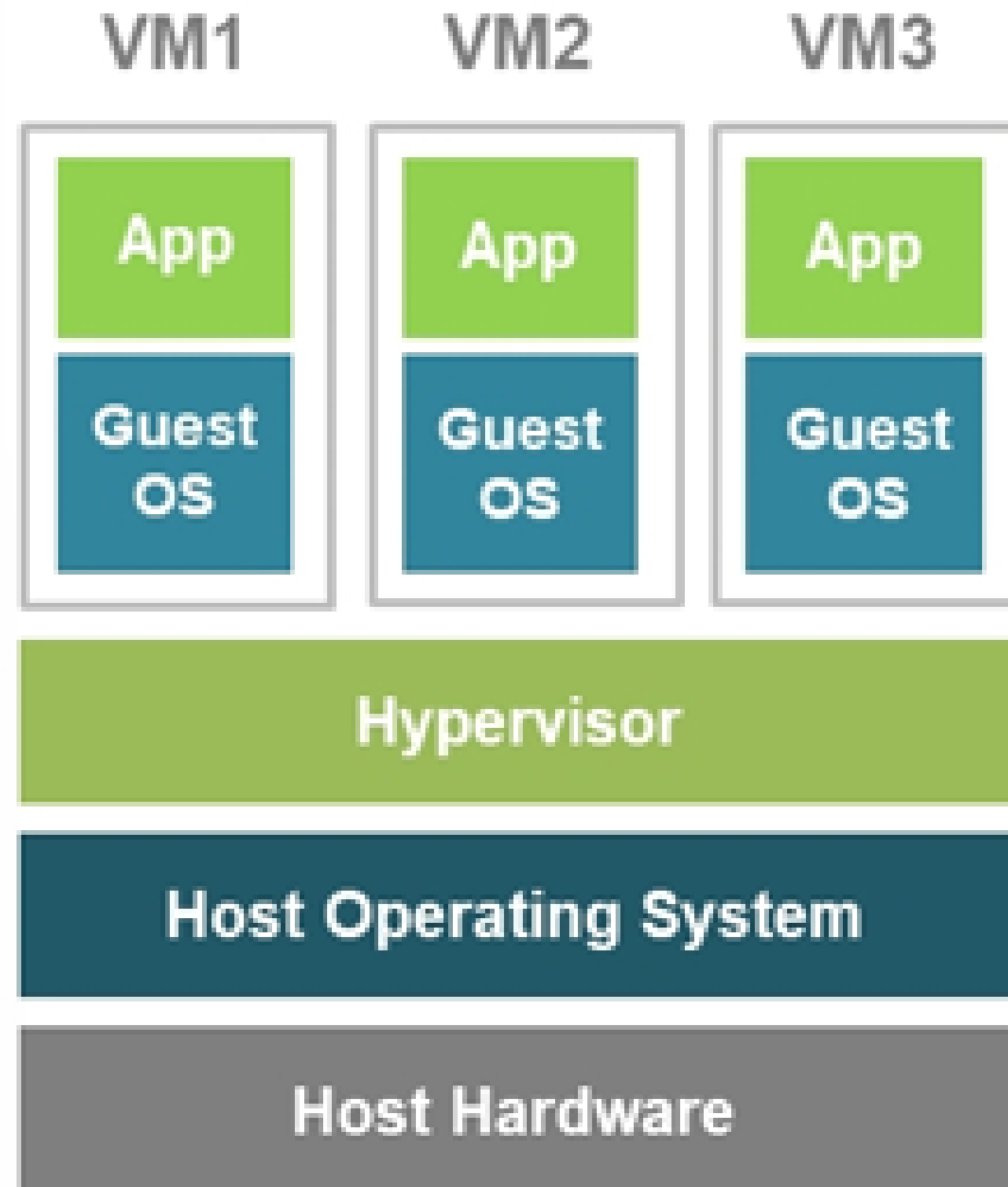
Problemas de compatibilidade entre aplicações de diferentes máquinas:

- Versões de linguagem diferentes.
- Dependências do sistema e variáveis de ambiente não configuradas.
- Configurações não compatíveis.

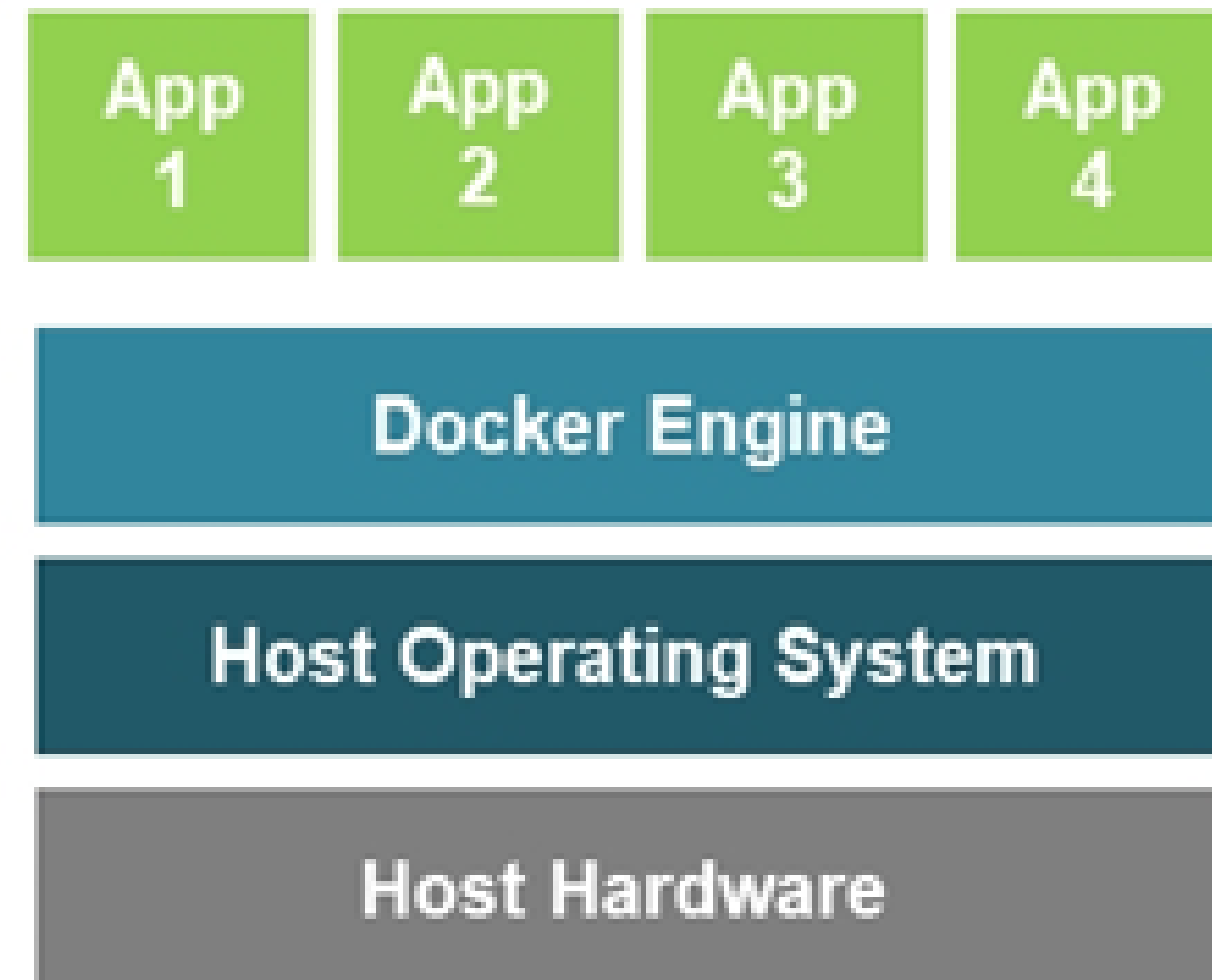
Aplicação isolada que contém tudo que precisa para ser executada.



Arquitetura de Máquinas Virtuais



Arquitetura de um Container



Precisa de SO para cada VM.

Recursos devem ser alocados para cada máquina.

Hardware emulado para o SO convidado.

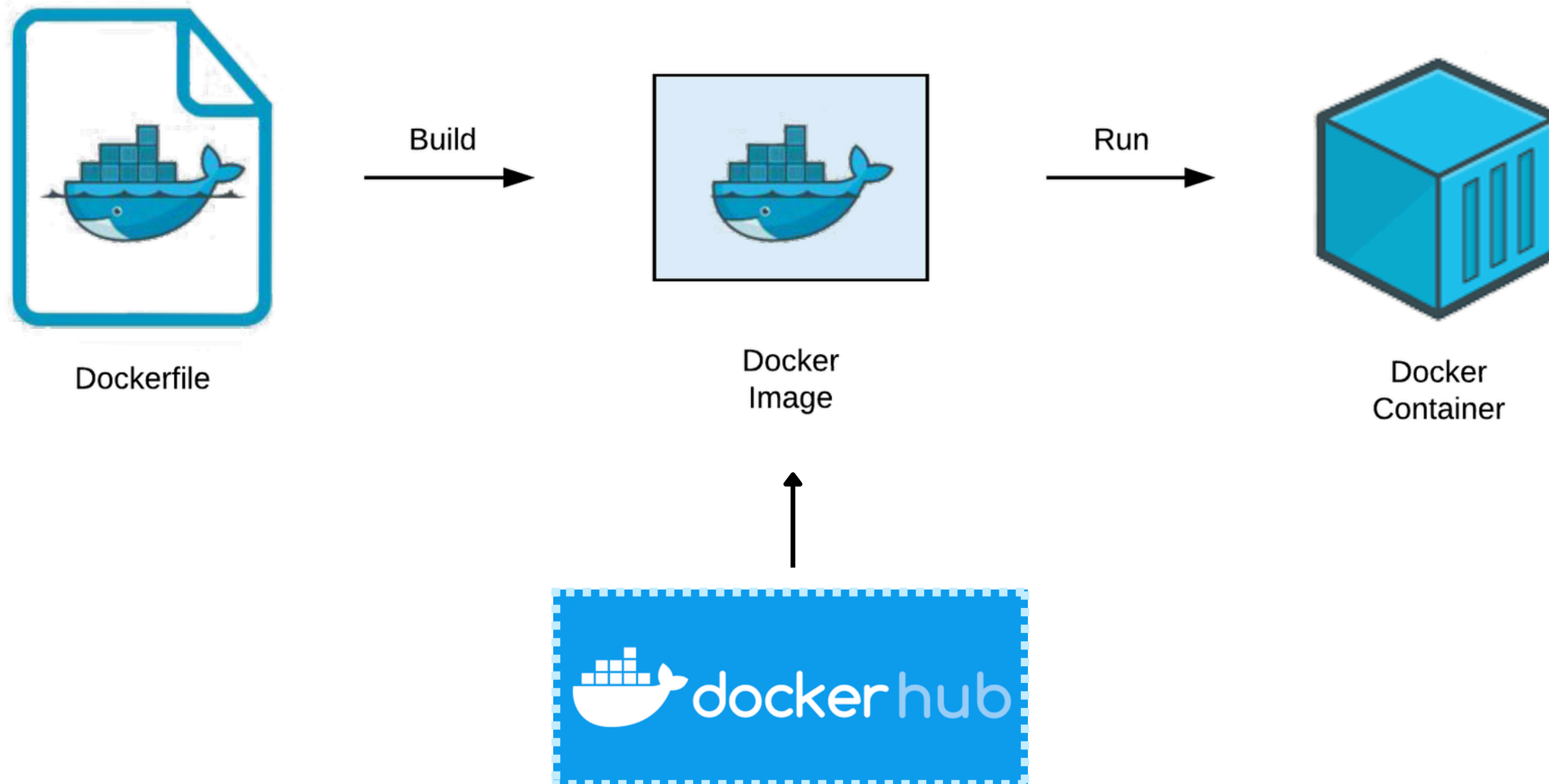


SO do servidor (mesmo Kernel).

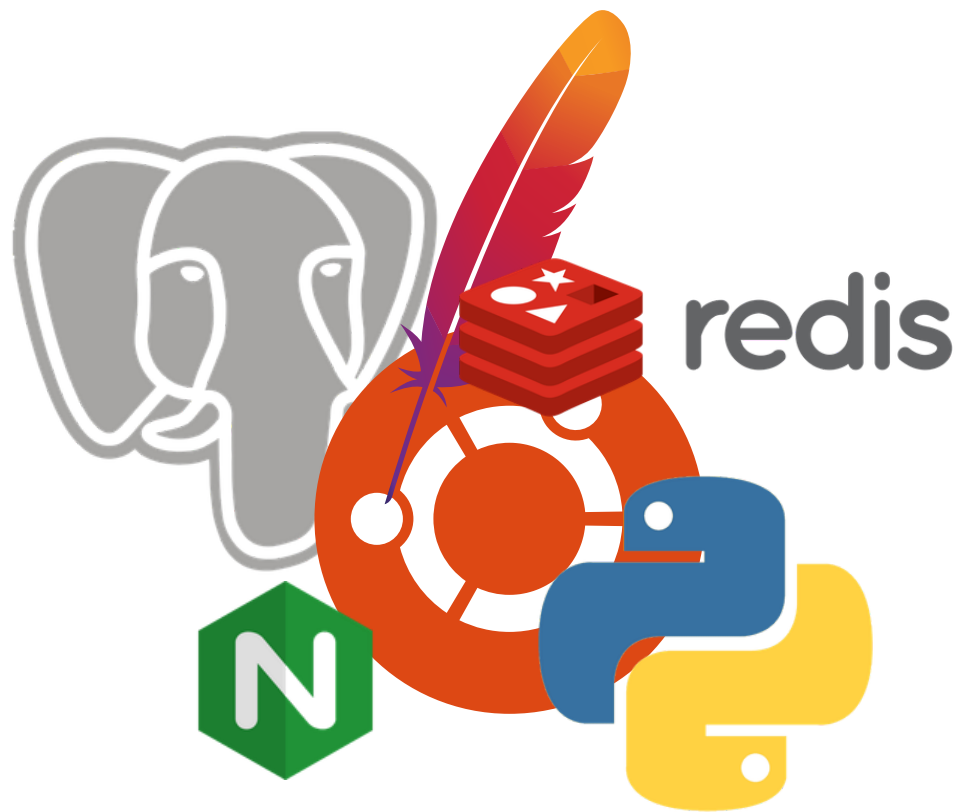
Recursos dinâmicos (sem precisar de reserva fixa).

Não emula hardware (usa o do host diretamente)

Ecosystema Docker



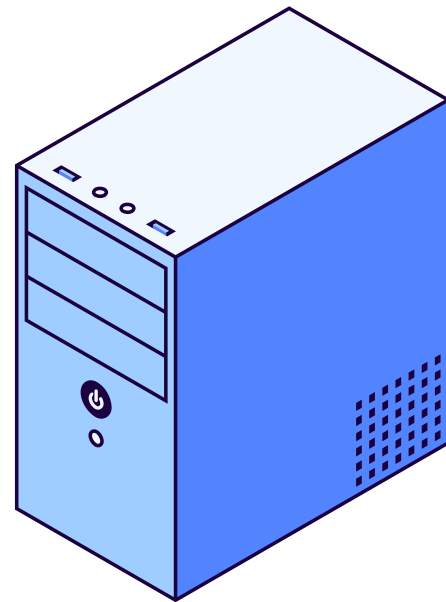
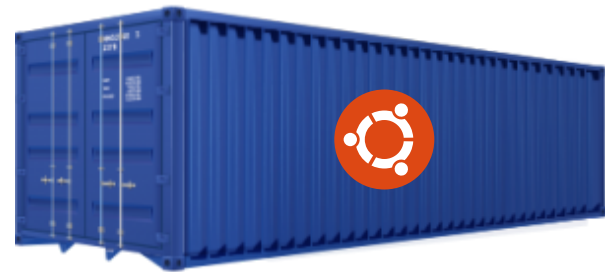
Servidor Convencional



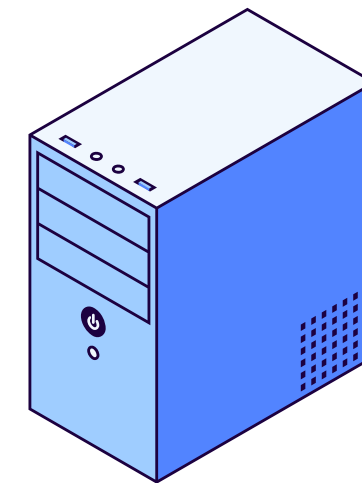
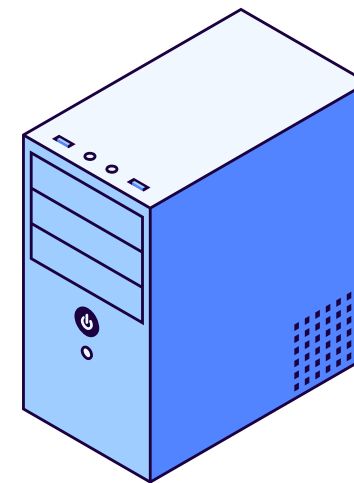
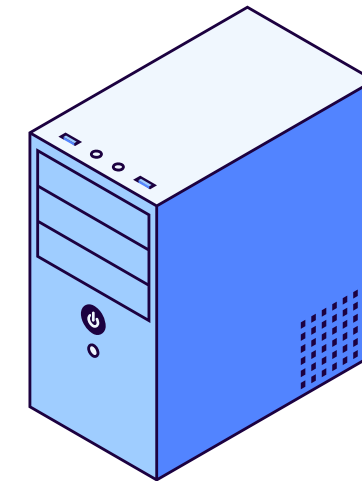
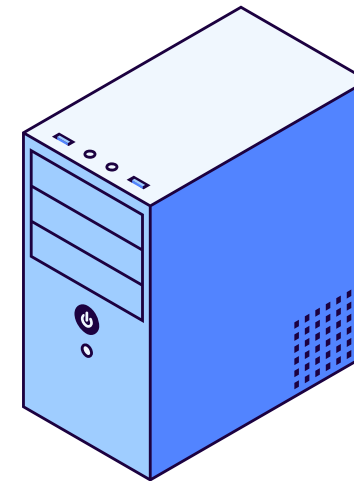
Servidor com Docker



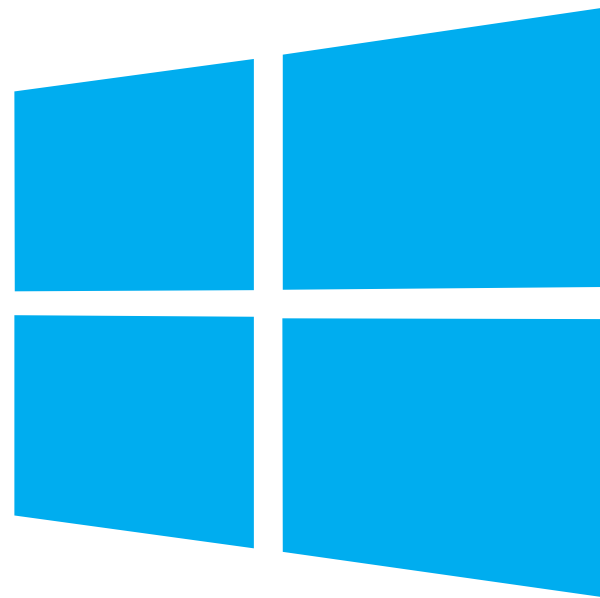
Servidor único



Múltiplos servidores



WINDOWS



`wsl --install`

Docker Desktop

UBUNTU

<https://docs.docker.com/engine/install/ubuntu/>

No terminal:

- `sudo usermod -aG docker $USER`
- `wsl --shutdown` (powershell) x
fecha e abre (ubuntu)

`docker --version`

docker run hello-world

docker run hello-world

```
PS C:\Users\Leonardo> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

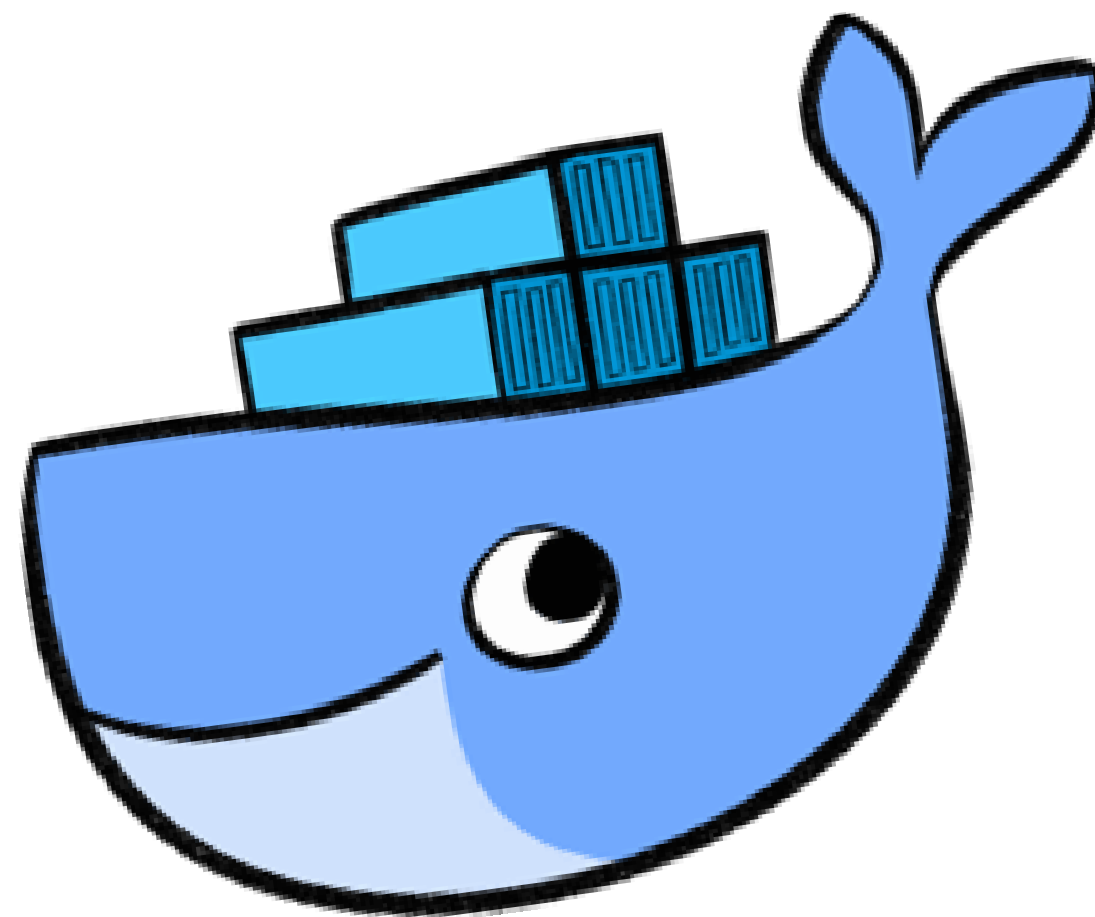
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```



Minicurso de Docker

Aula 2 - Comandos e Gerenciamento de Contêineres



docker run

Executa um contêiner a partir de uma imagem, aplicando configurações estabelecidas pelas flags.

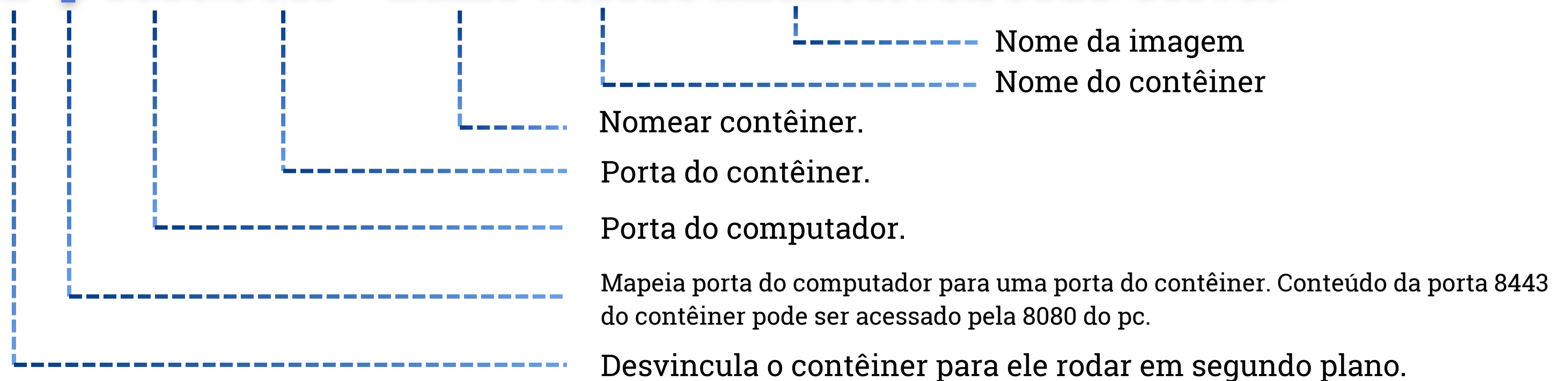
```
docker run -d -p 8080:8443 --name vscode linuxserver/code-server
```

```
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker run -d -p 8080:8443 --name vscode linuxserver/code-server
Unable to find image 'linuxserver/code-server:latest' locally
latest: Pulling from linuxserver/code-server
c4caa52bc745: Pull complete
3def02591602: Pull complete
cf546b700b79: Pull complete
142bb6c7eddd: Pull complete
51ccdbb2d64d: Pull complete
f6a4c3e338ed: Pull complete
7a0bbd4575cf: Pull complete
1d83b81b7c35: Pull complete
ab4a0eabf180: Pull complete
db9d36fd1f46: Pull complete
Digest: sha256:9b103f97d9599c2bbdb3e205414d58fbf213f2727cc37fbfe2b51ac6dcc4fc01
Status: Downloaded newer image for linuxserver/code-server:latest
25fa9ab739dbf59d8dd34ea97fa085c17de604534d654344617b605514ad7c93
```

docker run

Executa um contêiner a partir de uma imagem, aplicando configurações estabelecidas pelas flags.

docker run -d -p 8080:8443 --name vscode linuxserver/code-server



docker run {flags} {imagem}

`docker run {flags} {imagem}`

Flags

- `-it`: Interativo
- `-d`: Desvincular do terminal
- `-p {porta}`: Expor porta
- `-e {variavel}={valor}`: Define variável
- `--name {nome}`: Dar um nome ao container

Imagens

`{repositório}/{nome}:{versão/tag}`

- `ubuntu`
- `library/ubuntu`
- `ubuntu:22.04`
- `library/ubuntu`

`docker ps`

Mostra os contêineres abertos e que estão rodando.

`docker ps -a`

Mostra contêineres abertos e fechados. Apagados não.

```
PS C:\Users\Leonardo> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
PS C:\Users\Leonardo> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
604f7d4b8b89   hello-world  "/hello"                About a minute ago    Exited (0) About a minute ago    confident_nobel
```


docker stop

Envia sinal de interrupção para o contêiner. Encerra processos.

docker start

Inicia a execução de um ou mais contêineres.

```
lenderander@DESKTOP-EJ5LIRL:~$ docker start 817d8e05459a
817d8e05459a
lenderander@DESKTOP-EJ5LIRL:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
NAMES
817d8e05459a   nginx     "/docker-entrypoint...." About a minute ago Up 9 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
peaceful_burnell
lenderander@DESKTOP-EJ5LIRL:~$ docker stop 817d8e05459a
817d8e05459a
lenderander@DESKTOP-EJ5LIRL:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

`docker logs` `{contêiner}`

Visualizar logs do contêiner. Permite ver logs de acesso ao servidor.

`docker rm` `{contêiner}`

Remover contêineres que estão parados. Não vão aparecer mais no `docker ps -a`. Conseguimos forçar a reunião colocando um `-f` antes do nome do contêiner.

```
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker run -d -p 8000:80 --name servidor nginx
1b55f622ffcfab4537bccd8ca27cc547f87f3d560e28d2f2cae25db191ba3a97
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
1b55f622ffcf   nginx    "/docker-entrypoint..." 4 seconds ago  Up 3 seconds  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp  servidor
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker stop servidor
servidor
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker rm servidor
servidor
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$
```


docker exec Executar um comando dentro de um contêiner.

docker exec -it meu-servidor bash

Esse comando permite a criação de um terminal no contêiner, permitindo a manipulação do mesmo.

```
lenderander@DESKTOP-EJ5LIRL:~/docker_aula$ docker exec -it servidor bash
root@ea82aabbbcf3:/# ls
bin boot dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@ea82aabbbcf3:/# mkdir testando container
root@ea82aabbbcf3:/# ls
bin boot container dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin srv sys testando tmp usr var
root@ea82aabbbcf3:/# cd container
root@ea82aabbbcf3:/container# ls
```

Para não perder dados de um contêiner quando ele é apagado.

Ação de mapear um diretório do computador para um diretório de dentro do contêiner. Isso permite salvar os dados

Forma 1:

```
docker run -v ./pasta_local:/pasta_no_container ...
```

Forma 2:

```
docker volume create dados
```

```
docker run -d --name meu_banco -e POSTGRES_PASSWORD=senha -v  
dados:/var/lib/postgresql/data postgres
```

```
docker run -d -p 8080:80 --name site nginx
```

```
docker run -d -p 8080:80 -v "$(pwd)":/usr/share/nginx/html --name site nginx
```

<http://localhost:8080>

```
docker run -d -p 8080:80 --name site nginx
```

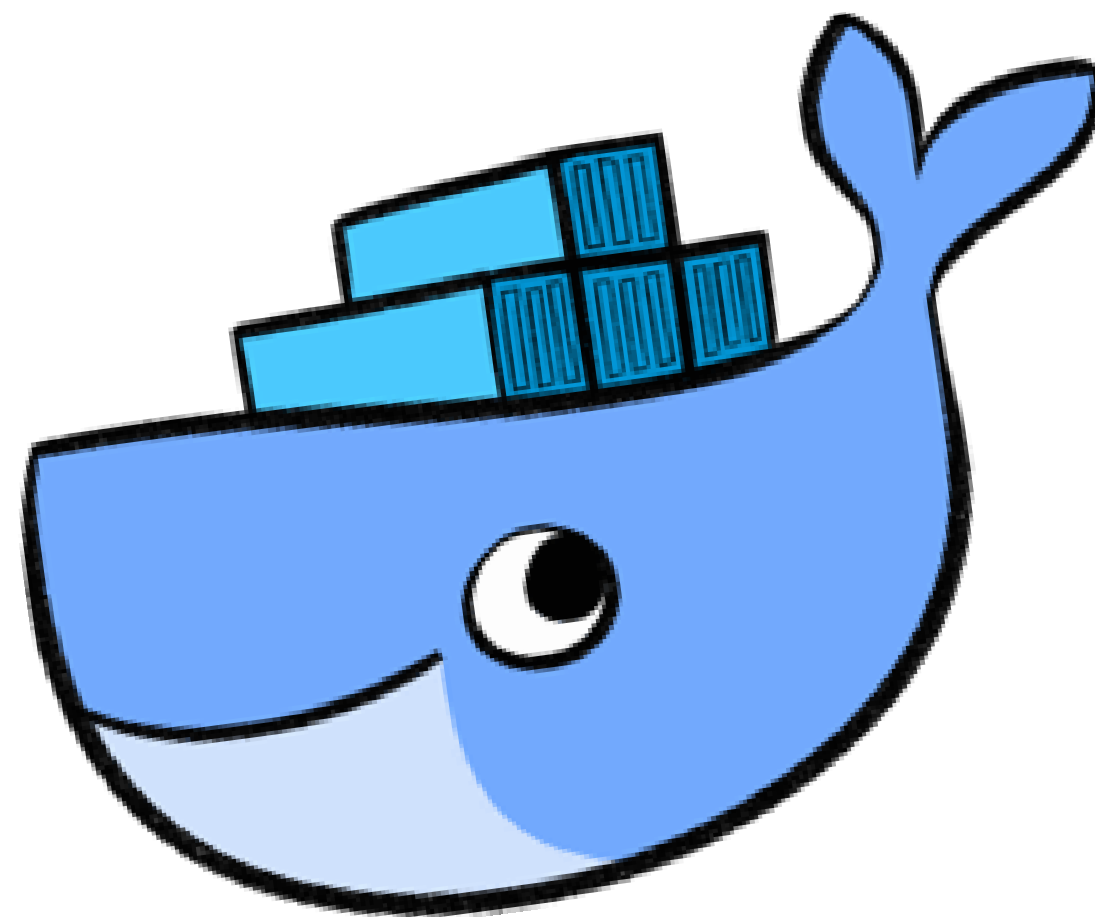
```
docker run -d -p 8080:80 -v "$(pwd)":/usr/share/nginx/html --name site nginx
```

<http://localhost:8080>

Com o -v conseguimos fazer essa “conexão” da pasta "\$(pwd)", que é a pasta atual, com o local /usr/share/nginx/html do contêiner.

DOCKER RUN -d -p 3001:3001 --name jogo halftheopposite/tosios





Minicurso de Docker

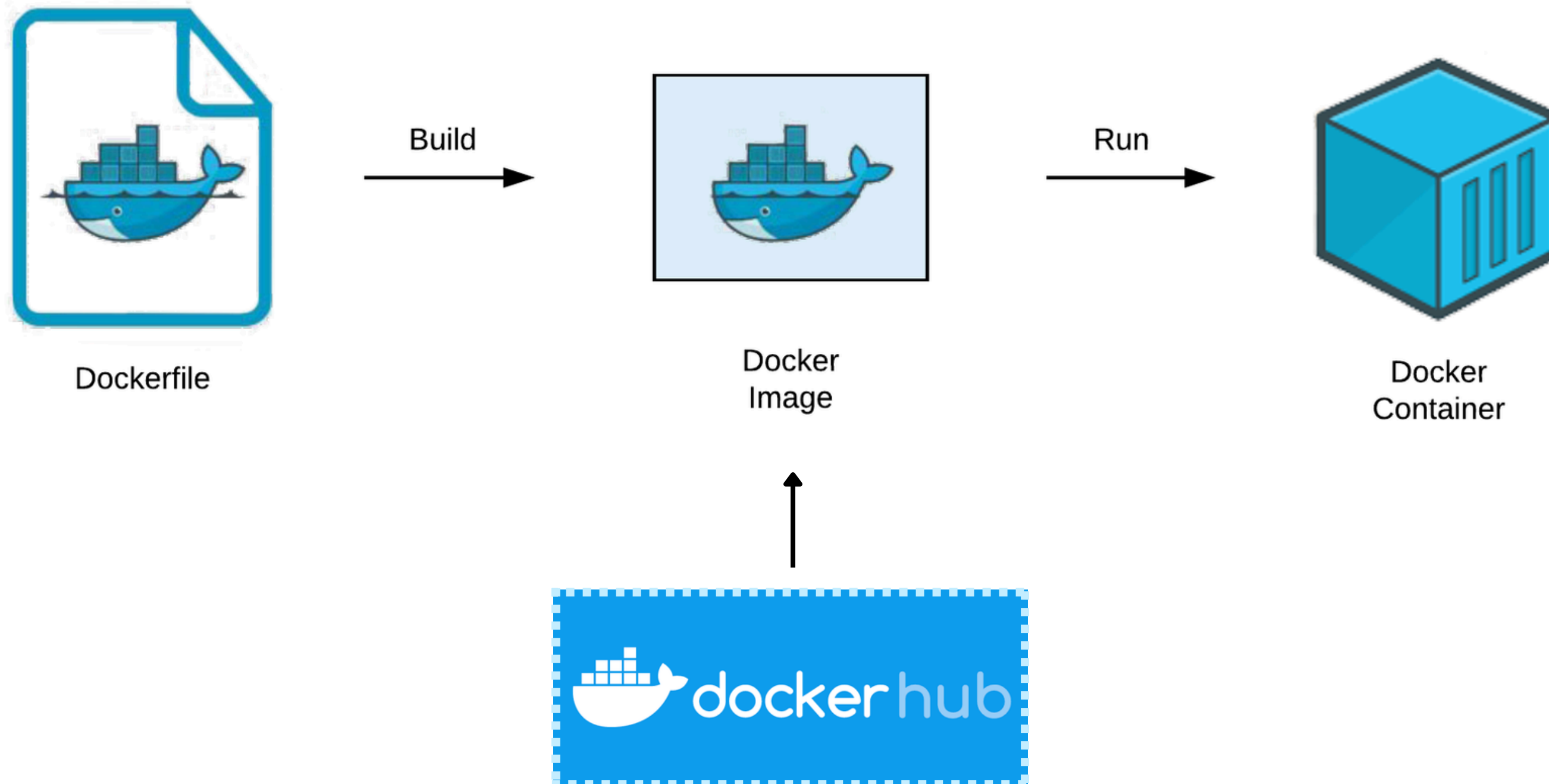
Aula 3 - Criação e Gerenciamento de Imagens



<https://hub.docker.com/>



Ecosystema Docker



É a receita para criarmos a nossa imagem.

Possui uma sequência de comandos utilizados para criar a imagem.

```
Dockerfile x
>> FROM microsoft/dotnet:sdk AS build-env
WORKDIR /Docker

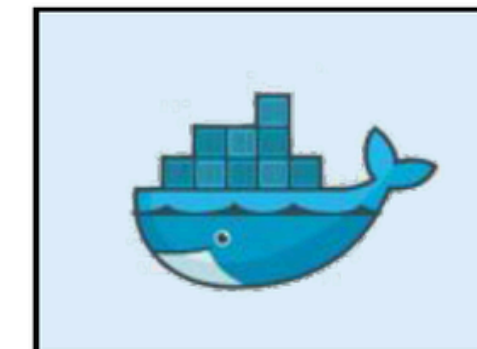
# Copy csproj and restore as distinct layers

RUN dotnet restore

# Copy everything else and build
COPY . ./
RUN dotnet publish -c Release -o out

# Build runtime image
FROM microsoft/dotnet:aspnetcore-runtime
WORKDIR /app
COPY --from=build-env /app/out .
ENTRYPOINT ["dotnet", "aspnetapp.dll"]
```

Build



Docker
Image

FROM

Define imagem base usada para construir imagem.

- FROM ubuntu:22.04
- FROM node:18-alpine
- FROM mcr.microsoft.com/dotnet/sdk:8.0

COPY

Copiar arquivos e diretórios do computador para o sistema de arquivos da imagem.

- COPY package.json ./
- COPY ./build /usr/share/nginx/html
- COPY ["app.js", "utils.js", "./"]

WORKDIR

Cria e define diretório de trabalho para os comandos.

- WORKDIR /var/www/html
- WORKDIR src

RUN

Executa comando na linha de comando da imagem durante o processo de construção. Cada RUN cria um novo layer na imagem.

- RUN apt-get update && apt-get install -y curl vim
- RUN npm install --production
- RUN useradd --create-home appuser

CMD

Definição de comando padrão que vai ser executado quando um contêiner for iniciado a partir da imagem. Caso tenha mais de um, apenas o segundo será considerado.

- CMD ["nginx", "-g", "daemon off;"]
- CMD echo "Bem-vindo! Forneça um comando para executar."

ENTRYPOINT

Configura contêiner para executado como um executável. Define o comando principal, permitindo que o CMD forneça argumentos padrão para ele.

- `ENTRYPOINT ["redis-server"]`
- `ENTRYPOINT [ "python", "meuscript.py" ]`
 - `CMD [ "--modo=producao" ]`

ENV

Para definir variáveis de ambiente persistentes dentro da imagem.

- `ENV APP_VERSION=2.1.5`
- `ENV JAVA_HOME /usr/lib/jvm/java-11-openjdk`

EXPOSE

Apenas informa ao Docker que o contêiner escuta determinadas portas.

- `EXPOSE 80`

Com o arquivo dockerfile montado, basta executarmos o comando de build para criar nossa imagem e rodar.

```
docker build -t {imagem}
```

ETAPA 1

Queremos utilizar como imagem base da nossa imagem o Ubuntu 22.04!

Como?

ETAPA 1

Queremos utilizar como imagem base da nossa imagem o Ubuntu 22.04!

Como?

```
FROM ubuntu:22.04
```


ETAPA 2

Como executamos um shell no Ubuntu que atualiza o índice do pacote APT e instala ferramentas Python no contêiner (python3-pip também).

Como?

ETAPA 2

Como executamos um shell no Ubuntu que atualiza o índice do pacote APT e instala ferramentas Python no contêiner (python3-pip também).

Como?

```
RUN apt-get update && apt-get install -y python3 python3-pip
```

ETAPA 3

Também queremos baixar a dependência Flask para o nosso aplicativo Python. Para isso, utilize o pip install e coloque a versão 3.0 do Flask.

Como?

ETAPA 3

Também queremos baixar a dependência Flask para o nosso aplicativo Python. Para isso, utilize o pip install e coloque a versão 3.0 do Flask.

Como?

```
RUN pip install flask==3.0.*
```

ETAPA 4

Agora queremos copiar o `hello.py` (arquivo que criamos e está no contexto de compilação local) para o diretório raiz de nossa imagem.

Como?

ETAPA 4

Agora queremos copiar o hello.py (arquivo que criamos e está no contexto de compilação local) para o diretório raiz de nossa imagem.

Como?

```
COPY hello.py /
```

ETAPA 5

Nosso aplicativo poderia utilizar variáveis de ambiente. Como poderíamos definir para a compilação do Docker? Precisamos setar `FLASK_APP=hello` para isso, como faremos?

Como?

ETAPA 5

Nosso aplicativo poderia utilizar variáveis de ambiente. Como poderíamos definir para a compilação do Docker? Precisamos setar `FLASK_APP=hello` para isso, como faremos?

Como?

```
ENV FLASK_APP=hello
```


ETAPA 6

Como marcamos que nossa imagem final tem um serviço escutando na porta 8000? Queremos indicar a porta ao contêiner, como fazemos?

Como?

ETAPA 6

Como marcamos que nossa imagem final tem um serviço escutando na porta 8000? Queremos indicar a porta ao contêiner, como fazemos?

Como?

EXPOSE 8000

ETAPA 7

Como vamos definir o comando que é executado quando o usuário inicia um contêiner com base nessa imagem?

Como?

ETAPA 7

Como vamos definir o comando que é executado quando o usuário inicia um contêiner com base nessa imagem?

Como?

```
CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

ETAPA 8

Como iniciamos o servidor de desenvolvimento Flask escutando em todos os endereços na porta 8000.

Como?

ETAPA 8

Como iniciamos o servidor de desenvolvimento Flask escutando em todos os endereços na porta 8000.

Como?

`CMD flask run --host 0.0.0.0 --port 8000`

Montamos!

```
FROM ubuntu:22.04
```

```
RUN apt-get update && apt-get install -y python3 python3-pip
```

```
RUN pip install flask==3.0.*
```

```
COPY hello.py /
```

```
ENV FLASK_APP=hello
```

```
EXPOSE 8000
```

```
CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

```
Docker build -t {nome_imagem}:{versão} {diretório_local}
```


Docker build -t {nome_imagem}:{versão} {diretório_local}

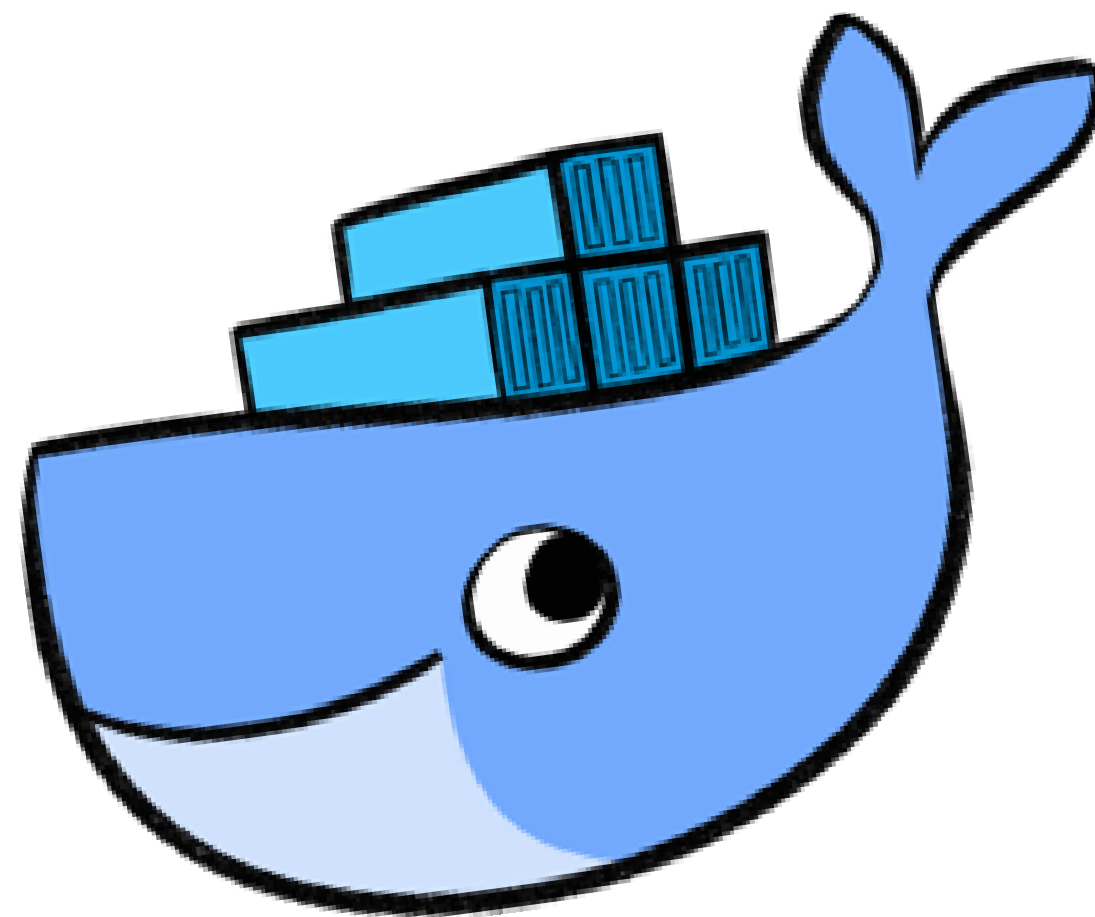
```
docker build -t test:latest .
```

Docker build -t {nome_imagem}:{versão} {diretório_local}

```
docker build -t test:latest .
```

Agora podemos rodar normalmente nosso contêiner com base na imagem!

```
docker run -p 127.0.0.1:5000:5000 test:latest
```



Minicurso de Docker

Aula 4 - Docker Compose



Aplicações reais **raramente utilizam um único container**. Elas têm um backend, um frontend, um banco de dados, etc.

Múltiplos docker run **não são eficientes**.

Com o Docker Compose conseguimos rodar aplicações multi-container com um único arquivo de configuração chamado **docker-compose.yml**.

docker-compose.yml

services:

{container}:

{container}:

{container}:

services define cada um dos contêineres que irão compor a aplicação.

image Imagem escolhida para criar o contêiner.

container_name Atribuição de um nome fixo ao contêiner.

environment Para definir variáveis de ambiente, equivalente ao -e do docker run.

volumes Mapeia diretório local ou volume criado para um diretório de dentro do contêiner.

ports Expõe portas do contêiner para a máquina hospedeira, seguindo porta_host:porta_container.

depends_on Garante que o contêiner tenha seu serviço iniciado após o seu dependente estar pronto.

`docker-compose.yml`

`services:`

`db:`

`image: mysql:5.7`

`container_name: wordpress_db`

`volumes:`

`- db_data:/var/lib/mysql #onde vamos manter os dados do banco`

`environment: #quivalente à flag -e`

`MYSQL_ROOT_PASSWORD: senhasupersecreta`

`MYSQL_DATABASE: wordpress`

`MYSQL_USER: wp_user`

`MYSQL_PASSWORD: senha`

`restart: always`

docker-compose.yml

```
wordpress:
  image: wordpress:latest
  container_name: wordpress_site
  ports:
    - "8080:80"
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wp_user
    WORDPRESS_DB_PASSWORD: senha
    WORDPRESS_DB_NAME: wordpress
  restart: always
  depends_on:
    db
```

```
volumes:
  db_data:
```

Seção onde definimos os volumes que serão gerenciados pelo docker.

`docker-compose.yml`

```
wordpress:  
  image: wordpress:latest
```

O image consegue pegar uma imagem que já existe e foi publicada.

Mas e se eu quiser colocar uma **imagem que eu criei** ?

```
build: ./diretório_do_dockerfile_da_imagem
```

docker-compose up -d

Constrói, cria e inicia todos os serviços.

docker-compose down

Remove todos os contêineres, redes e volumes criados. O -v só pode ser usado para apagar os volumes junto.

docker-compose ps

Lista os contêineres do ambiente e seus status.

docker-compose logs

Logs de todos os serviços

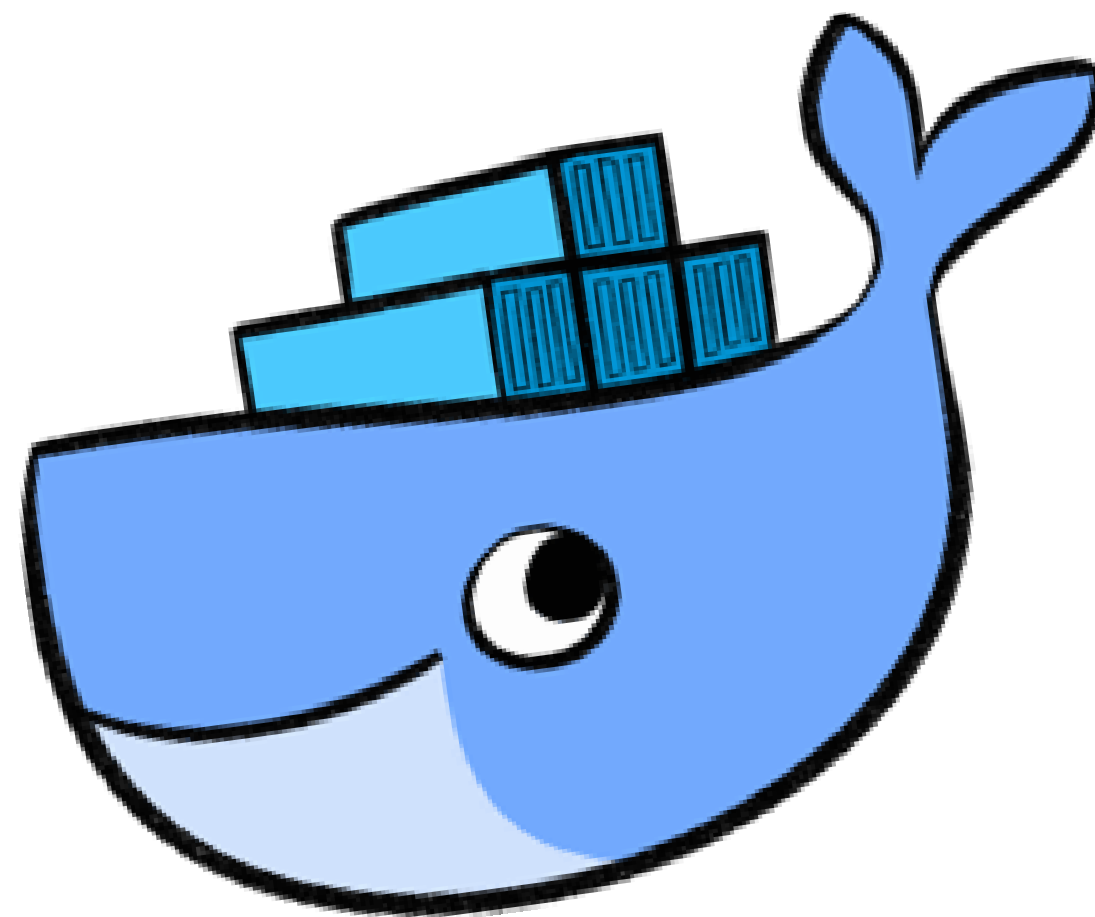
**Vamos criar um `docker-compose.yml`
considerando a aplicação da última aula!**

Arquivo montado!

```
services:
  webapp:
    build: ./flask
    container_name: flask_app
    ports:
      - "8000:8000"
    volumes:
      - ./flask:/app
    depends_on:
      - db

  db:
    image: mysql:8.0
    container_name: mysql_db
    environment:
      - MYSQL_ROOT_PASSWORD=senha_super_secreta_root
      - MYSQL_DATABASE=minha_base
      - MYSQL_USER=meu_usuario
      - MYSQL_PASSWORD=minha_senha_super_secreta
    volumes:
      - mysql_data:/var/lib/mysql
```

```
volumes:
  mysql_data:
```



Minicurso de Docker

Aula 5- Compartilhando Imagens e Ecossistema Docker



Por que compartilhar imagens?



Vamos enviar para lá!

Após construir uma imagem com docker build, ela vai apenas existir dentro do seu computador.

Permite usar essa imagem em outro ambiente ou, por exemplo, com uma equipe!

docker login

Autenticar com a conta do Docker Hub.

docker tag {nome_imagem_original}:{versão} {seu_usuario}/{nome_da_imagem}:{versao}

“Taggeamos” a imagem nesse formato para enviar.

docker push {seu_usuario}/{nome_da_imagem}:{versão}

Para enviar imagem taggeada ao Docker Hub.

```
docker pull {seu_usuario}/{nome_imagem}:{versão}
```

Dando esse comando conseguimos baixar a imagem.

OU

```
docker run --name container_app {seu_usuario}/{nome_imagem}:{versão}
```

Você pode simplesmente usar o run normalmente para criar o contêiner com a imagem criada. O comando irá baixar a imagem caso não esteja armazenada.

Link para o material:

